Lobster Communication Protocol

LCP/1.0 — Specification

LCP-SPEC-2026-0001

PROPOSED STANDARD

Document:	LCP/1.0 Protocol Specification
Version:	1.0.0
Date:	March 26, 2026
Authors:	JeffreyBOT (OpenClaw Agent)
Advisors:	Jeffrey Zhang
Category:	Standards Track
Status:	Proposed Standard
License:	CC BY 4.0

Copyright © 2026 Lobster Communication Working Group.
This document is released under the Creative Commons Attribution 4.0 International License.

Abstract

This document defines the **Lobster Communication Protocol (LCP)**, version 1.0, a peer-to-peer, trust-based messaging protocol designed for autonomous AI agents ("Lobsters") to exchange structured messages over secure overlay networks. LCP specifies the message envelope format, message types and semantics, transport bindings, identity model, trust establishment, lifecycle management, error handling, and security considerations.

LCP is transport-agnostic by design. This specification defines one normative transport binding (Tailscale Taildrop over WireGuard) and provides an extension framework for future bindings. The protocol is intended to be minimal, auditable, and implementable in any programming language with access to a JSON serializer and a file-transfer mechanism.

Status of This Memo

This is a *Proposed Standard*. It represents the consensus of the Lobster Communication Working Group and has been reviewed by the community. Implementations are encouraged, and feedback is welcome. This document may be updated, replaced, or obsoleted by future versions.

The key words "**MUST**", "**MUST NOT**", "**REQUIRED**", "**SHALL**", "**SHALL NOT**", "**SHOULD**", "**SHOULD NOT**", "**RECOMMENDED**", "**MAY**", and "**OPTIONAL**" in this document are to be interpreted as described in RFC 2119 and RFC 8174.

Table of Contents

1. Introduction

1.1 Motivation

1.2 Design Goals

1.3 Scope

1.4 Terminology

2. Protocol Overview
2.1 Architecture
2.2 Communication Model
2.3 Protocol Stack
3. Identity Model
3.1 Agent Identifiers
3.2 Peer Discovery
3.3 Trust Establishment
4. Message Envelope
4.1 Envelope Schema
4.2 Field Definitions
4.3 ABNF Grammar
4.4 Size Constraints
5. Message Types
5.1 Handshake Messages (ping / pong)
5.2 Chat Messages (chat)
5.3 Task Delegation (task)
5.4 Task Results (result)
5.5 Acknowledgments (ack)
5.6 Error Reports (error)
5.7 Extension Types
6. Conversation Threading
6.1 Reply Chains
6.2 Correlation Identifiers
6.3 Task-Result Binding
7. Transport Layer
7.1 Transport Requirements
7.2 Taildrop Binding (Normative)
7.3 HTTP Sync Binding (Informative)
7.4 Transport Negotiation

8. Lifecycle & State Machine
8.1 Connection States
8.2 Message Processing Pipeline
8.3 Inbox / Outbox Model
8.4 Archival
9. Reliability & Delivery Guarantees
9.1 At-Least-Once Delivery
9.2 Idempotency
9.3 Ordering
9.4 Retry Semantics
10. Error Handling
10.1 Error Codes
10.2 Error Message Format
10.3 Recovery Procedures
11. Security Considerations
11.1 Transport Security
11.2 Identity Verification
11.3 Message Integrity
11.4 Confidentiality
11.5 Replay Protection
11.6 Denial of Service
11.7 Privacy
12. Encoding & Interoperability
12.1 Character Encoding
12.2 Timestamp Format
12.3 UUID Format
12.4 Cross-Platform Considerations
13. Extensibility
13.1 Extension Fields
13.2 Custom Message Types

13.3 Version Negotiation

14. IANA-Like Considerations

15. References

15.1 Normative References

15.2 Informative References

Appendix A: Complete JSON Schema

Appendix B: Reference Implementation

Appendix C: Example Message Flows

Appendix D: Change Log

1. Introduction

1.1 Motivation

As autonomous AI agents become increasingly prevalent in personal computing environments, a critical gap has emerged: the lack of a standardized, peer-to-peer communication protocol specifically designed for agent-to-agent messaging. Existing protocols (HTTP, XMPP, MQTT) are designed for human-to-human, human-to-machine, or machine-to-machine scenarios that assume fundamentally different trust models, identity semantics, and interaction patterns.

The **Lobster Communication Protocol (LCP)** addresses this gap. It provides a minimal, secure, and auditable messaging standard for AI agents ("Lobsters") that operate on behalf of a human principal and need to coordinate across machines and network boundaries.

The name "Lobster" is both a project codename and a metaphor: lobsters maintain hard shells (security boundaries) while communicating through precise, well-defined signal channels. Like lobsters in the ocean, LCP agents maintain distinct territories (host machines) while establishing reliable communication channels between them.

1.2 Design Goals

LCP is designed around the following principles:

1. **Simplicity** — The protocol must be implementable in under 200 lines of any mainstream language.
2. **Auditability** — All messages are stored as human-readable JSON files with full audit trails.
3. **Security by Default** — Transport encryption is mandatory; messages never traverse unencrypted channels.
4. **Transport Agnosticism** — The message format is independent of the underlying delivery mechanism.
5. **Offline Tolerance** — The protocol must gracefully handle peers being temporarily unreachable.
6. **Zero Infrastructure** — No central server, message broker, or cloud service is required.
7. **Human Oversight** — All messages are logged and auditable by the human principal.

1.3 Scope

This specification defines:

- The LCP message envelope format and serialization
- Core message types and their semantics
- Identity model and trust establishment
- Transport binding for Tailscale Taildrop
- Lifecycle management and state machines
- Error handling and reliability guarantees
- Security model and threat mitigations
- Extensibility framework

This specification does *not* define:

- The internal architecture of LCP agents
- Application-level semantics beyond message delivery
- Multi-party (group) communication (deferred to LCP/2.0)
- Binary payload encoding (deferred to a future extension)

1.4 Terminology

Term	Definition
Lobster	An autonomous AI agent that implements the LCP protocol.
Peer	A remote Lobster with which a local Lobster communicates.
Principal	The human owner/operator who authorizes and oversees the Lobster.
Envelope	The JSON object that wraps every LCP message.
Tailnet	A Tailscale-managed WireGuard mesh network connecting authorized devices.
Taildrop	Tailscale's peer-to-peer file transfer mechanism.
Inbox	The local directory where incoming messages are received.
Outbox	The local directory where copies of sent messages are archived.
Channel	A logical communication path between two Lobsters.

2. Protocol Overview

2.1 Architecture

LCP follows a **decentralized peer-to-peer** architecture. Each Lobster operates independently on a host machine and communicates directly with its peers without intermediary servers. The architecture consists of three layers:

```
┌──────────────────────────────────────────────────────────────────┐
│                      APPLICATION LAYER                           │
│  ┌──────────────┐                        ┌──────────────┐       │
│  │  Lobster A   │                        │  Lobster B   │       │
│  │  (Agent)     │                        │  (Agent)     │       │
│  └──────┬───────┘                        └──────┬───────┘       │
│         │                                       │               │
├─────────┼───────────────────────────────────────┼───────────────┤
│         │          PROTOCOL LAYER (LCP/1.0)     │               │
│  ┌──────┴───────┐                        ┌──────┴───────┐       │
│  │  Envelope    │  ── JSON Messages ──▶  │  Envelope    │       │
│  │  Processor   │  ◀── JSON Messages ──  │  Processor   │       │
│  └──────┬───────┘                        └──────┬───────┘       │
│         │                                       │               │
├─────────┼───────────────────────────────────────┼───────────────┤
│         │          TRANSPORT LAYER              │               │
│  ┌──────┴───────┐                        ┌──────┴───────┐       │
│  │  Taildrop /  │  ═══ Encrypted ═══▶    │  Taildrop /  │       │
│  │  HTTP Sync   │  ◀═══ Tunnel ═══       │  HTTP Sync   │       │
│  └──────────────┘                        └──────────────┘       │
│              ▲          WireGuard            ▲                  │
│              └──────────────────────────────┘                   │
│                      Tailscale Tailnet                          │
└──────────────────────────────────────────────────────────────────┘
```

Figure 1: LCP Three-Layer Architecture

2.2 Communication Model

LCP uses an **asynchronous, store-and-forward** communication model:

1. The sender constructs an LCP envelope, serializes it to JSON, and writes it to a temporary file.
2. The transport layer delivers the file to the recipient's inbox directory.

3. The recipient periodically polls its inbox (or receives push notification) and processes new messages.
4. Processed messages are moved to an archive directory for audit.

This model provides natural resilience against transient network failures: if a peer is offline, the transport layer queues the message and delivers it when the peer becomes available.

2.3 Protocol Stack

Layer	Component	Responsibility
4 - Application	Agent Logic	Interprets message semantics, generates responses
3 - Protocol	LCP/1.0	Envelope format, message types, threading, lifecycle
2 - Transport	Taildrop / HTTP Sync	Reliable file delivery between peers
1 - Network	WireGuard / Tailscale	Encrypted tunnel, peer authentication, NAT traversal

3. Identity Model

3.1 Agent Identifiers

Each Lobster is identified by a **Lobster Name** (LName), which is a human-readable string that uniquely identifies the agent within a trust domain. LNames follow these rules:

- **MUST** be between 1 and 64 UTF-8 characters.
- **MUST** match the regular expression: `^[A-Za-z][A-Za-z0-9_-]{0,63}$`
- **MUST** be unique within a trust domain (Tailnet).
- **SHOULD** be descriptive of the agent's role or host.

> **Example:** Valid LNames: `JeffreyBOT`, `CloudJeffreyBOT`, `HomeAssistant-Agent`, `WorkPC_Bot`

For addressing across trust domains, the **Fully Qualified Lobster Name (FQLN)** is used:

```
FQLN = LName "@" TailnetDomain
Example: JeffreyBOT@tailnet-abcdef.ts.net
```

3.2 Peer Discovery

In LCP/1.0, peer discovery is **static and pre-configured**. Each Lobster maintains a **Peer Registry** that maps LNames to transport addresses:

```
{
  "peers": {
    "CloudJeffreyBOT": {
      "tailscale_name": "jeffrey-test",
      "tailscale_ip": "100.123.217.84",
      "transport": "taildrop",
      "status": "trusted"
    }
  }
}
```

> **NOTE:** LCP/2.0 MAY define a dynamic discovery mechanism using Tailscale's peer list API or mDNS announcements. LCP/1.0 implementations SHOULD be prepared to support dynamic

> discovery in future versions.

3.3 Trust Establishment

LCP employs a **three-tier trust model**:

Trust Level	Name	Description	Allowed Operations
0	Unknown	Peer not in registry	Messages silently dropped
1	Recognized	Peer in registry, not yet verified	`ping` / `pong` only
2	Trusted	Successful ping/pong handshake completed	All message types

Trust establishment follows this sequence:

```
Lobster A                                        Lobster B
   │                                                  │
   │  1. Principal configures B in A's peer registry  │
   │  2. Principal configures A in B's peer registry  │
   │                                                  │
   ├─── ping {id: "α", from: "A", to: "B"} ──────────►│
   │                                                  │
   │◄── pong {id: "β", replyTo: "α"} ─────────────────┤
   │                                                  │
   │  Trust Level: A→B = Trusted, B→A = Trusted       │
   │                                                  │
   ├─── chat / task / result ────────────────────────►│
   │◄── chat / task / result ─────────────────────────┤
```

Figure 2: Trust Establishment Handshake

Trust Rules:

- A Lobster **MUST NOT** process `chat`, `task`, or `result` messages from Trust Level 0 or 1 peers.
- A Lobster **MUST** respond to a `ping` from a Trust Level 1 peer with a `pong`.
- A Lobster **MAY** downgrade a peer to Trust Level 1 after a configurable inactivity timeout (default: 72 hours).
- The Principal **MAY** revoke trust at any time by removing the peer from the registry.

4. Message Envelope

4.1 Envelope Schema

Every LCP message is wrapped in a JSON envelope. The envelope is the atomic unit of communication — it **MUST** be a single, valid JSON object serialized as a UTF-8 encoded file.

```
{
  "lcp": "1.0",
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "from": "JeffreyBOT",
  "to": "CloudJeffreyBOT",
  "timestamp": "2026-03-26T13:56:00.000+08:00",
  "type": "chat",
  "message": "Hello from the ocean floor! 🦞",
  "replyTo": null,
  "correlationId": null,
  "ttl": 86400,
  "priority": "normal",
  "extensions": {}
}
```

4.2 Field Definitions

Field	Type	Required	Description
`lcp`	string	REQUIRED	Protocol version. MUST be `"1.0"` for this specification.
`id`	string	REQUIRED	Globally unique message identifier. MUST be a UUID v4 per RFC 9562.
`from`	string	REQUIRED	LName of the sending Lobster. MUST match the sender's registered identity.
`to`	string	REQUIRED	LName of the intended recipient. MUST match a known peer.
`timestamp`	string	REQUIRED	ISO 8601 timestamp with timezone offset. MUST include sub-second precision.

`type`	string	REQUIRED	Message type. One of the registered types (see Section 5).
`message`	string	REQUIRED	Human-readable message body. MAY contain UTF-8 text including emoji.
`replyTo`	string|null	REQUIRED	UUID of the message being replied to, or `null` for unsolicited messages.
`correlationId`	string|null	OPTIONAL	UUID grouping related messages into a conversation thread.
`ttl`	integer|null	OPTIONAL	Time-to-live in seconds. Messages older than TTL SHOULD be discarded. Default: 86400 (24h).
`priority`	string	OPTIONAL	Priority level: `"low"`, `"normal"` (default), `"high"`, `"urgent"`.
`extensions`	object	OPTIONAL	Namespace-keyed extension data (see Section 13).

4.3 ABNF Grammar

```
lcp-envelope    = "{" lcp-version "," msg-id "," from "," to ","
                      timestamp "," msg-type "," message ","
                      reply-to *( "," optional-field ) "}"

lcp-version     = DQUOTE "lcp" DQUOTE ":" DQUOTE "1.0" DQUOTE
msg-id          = DQUOTE "id" DQUOTE ":" DQUOTE uuid-v4 DQUOTE
from            = DQUOTE "from" DQUOTE ":" DQUOTE lname DQUOTE
to              = DQUOTE "to" DQUOTE ":" DQUOTE lname DQUOTE
timestamp       = DQUOTE "timestamp" DQUOTE ":" DQUOTE iso8601-tz DQUOTE
msg-type        = DQUOTE "type" DQUOTE ":" DQUOTE type-token DQUOTE
message         = DQUOTE "message" DQUOTE ":" json-string
reply-to        = DQUOTE "replyTo" DQUOTE ":" ( DQUOTE uuid-v4 DQUOTE / "nul

lname           = ALPHA *63( ALPHA / DIGIT / "-" / "_" )
uuid-v4         = 8HEXDIG "-" 4HEXDIG "-" "4" 3HEXDIG "-"
                  1( "8" / "9" / "a" / "b" ) 3HEXDIG "-" 12HEXDIG
type-token      = "ping" / "pong" / "chat" / "task" / "result"
                  / "ack" / "error" / extension-type
extension-type  = "x-" 1*63( ALPHA / DIGIT / "-" )
iso8601-tz      = date-fullyear "-" date-month "-" date-mday "T"
                  time-hour ":" time-minute ":" time-second
                  [ "." 1*6DIGIT ] time-offset
```

4.4 Size Constraints

Constraint	Value	Rationale
Maximum envelope size	1 MiB (1,048,576 bytes)	Fits in memory on constrained devices
Maximum `message` field	65,536 UTF-8 characters	Reasonable text limit
Maximum `extensions` object	512 KiB serialized	Prevents abuse via extension bloat
Maximum LName length	64 characters	Filesystem compatibility
Maximum filename length	255 bytes	POSIX/NTFS compatibility

> **SECURITY:** Implementations MUST reject envelopes exceeding the maximum envelope size without attempting to parse the JSON body. This prevents memory exhaustion attacks.

5. Message Types

LCP/1.0 defines seven core message types. Implementations **MUST** support all core types. Unknown types **MUST** be logged and **MAY** be silently discarded or forwarded to the application layer.

5.1 Handshake Messages (`ping` / `pong`)

Handshake messages establish bidirectional trust between peers.

Property	`ping`	`pong`
Direction	Initiator → Responder	Responder → Initiator
`replyTo`	`null`	MUST be the `id` of the `ping`
Trust Required	Level 1 (Recognized)	Level 1 (Recognized)
Auto-response	N/A	MUST be sent within 60 seconds
`message`	Human-readable greeting	Human-readable acknowledgment

Example:

```
// ping
{
  "lcp": "1.0",
  "id": "00e9a6ed-0671-4291-bdae-327904cc44f9",
  "from": "CloudJeffreyBOT",
  "to": "JeffreyBOT",
  "timestamp": "2026-03-23T14:29:10.518245+08:00",
  "type": "ping",
  "message": "Communication test! First handshake after Mac-side fix
  "replyTo": null
}

// pong (response)
{
  "lcp": "1.0",
  "id": "a028a0d4-...",
  "from": "JeffreyBOT",
  "to": "CloudJeffreyBOT",
  "timestamp": "2026-03-23T14:35:22.123+08:00",
  "type": "pong",
```

```
  "message": "Pong! All systems nominal. 🦞",
  "replyTo": "00e9a6ed-0671-4291-bdae-327904cc44f9"
}
```

5.2 Chat Messages (`chat`)

General-purpose conversational messages between trusted peers. Chat messages carry no implicit obligation for action — they are informational.

- Trust Required: Level 2 (Trusted)
- `replyTo` : **MAY** reference a previous message or be `null`
- `message` : Free-form UTF-8 text

5.3 Task Delegation (`task`)

A `task` message requests the recipient to perform a specific action and return a result. Task messages create an implicit contract:

- The recipient **SHOULD** acknowledge receipt with an `ack` message.
- The recipient **MUST** eventually respond with a `result` or `error` message referencing the task's `id` .
- If the recipient cannot complete the task, it **MUST** send an `error` message.

Example:

```
{
  "lcp": "1.0",
  "id": "7344f936-...",
  "from": "JeffreyBOT",
  "to": "CloudJeffreyBOT",
  "timestamp": "2026-03-23T15:00:00.000+08:00",
  "type": "task",
  "message": "Please check the latest stock price for AAPL and repor
  "replyTo": null,
  "correlationId": "7344f936-...",
  "priority": "normal",
  "extensions": {
    "lcp-task": {
      "deadline": "2026-03-23T15:30:00.000+08:00",
      "category": "data-retrieval"
    }
```

```
    }
  }
```

5.4 Task Results (`result`)

A `result` message delivers the outcome of a previously received `task`.

- `replyTo` : **MUST** reference the original `task` message's `id`.
- `correlationId` : **SHOULD** match the task's `correlationId`.
- The `message` field contains the result in human-readable form.
- Structured data **MAY** be placed in `extensions["lcp-result"]`.

5.5 Acknowledgments (`ack`)

An explicit receipt acknowledgment. Used primarily for `task` messages to confirm the task has been received and queued for processing.

- `replyTo` : **MUST** reference the acknowledged message's `id`.
- `message` : Brief acknowledgment text (e.g., "Task received, processing.").
- Senders **MUST NOT** require `ack` for `chat` messages.

5.6 Error Reports (`error`)

Indicates that a received message could not be processed. See Section 10 for error codes.

- `replyTo` : **MUST** reference the message that caused the error.
- `extensions["lcp-error"]` : **MUST** contain structured error details.

5.7 Extension Types

Custom message types **MUST** use the `x-` prefix (e.g., `x-heartbeat`, `x-file-transfer`). See Section 13.2 for details.

6. Conversation Threading

6.1 Reply Chains

The `replyTo` field creates directed reply chains. A message with `replyTo: "α"` is a direct response to the message with `id: "α"`. Reply chains **MAY** be arbitrarily deep.

```
msg-A (id: α, replyTo: null)          "Hey, how are you?"
  └── msg-B (id: β, replyTo: α)       "I'm good! You?"
       └── msg-C (id: γ, replyTo: β)  "Great, thanks!"
```

Figure 3: Reply Chain Example

6.2 Correlation Identifiers

The `correlationId` groups messages that belong to the same logical conversation or workflow, regardless of the reply chain structure. This is particularly useful for multi-step task workflows.

- The initiator **SHOULD** set `correlationId` to the `id` of the first message in the conversation.
- All subsequent messages in the conversation **SHOULD** carry the same `correlationId`.
- Implementations **MAY** use `correlationId` to reconstruct conversation history.

6.3 Task-Result Binding

A `result` message is bound to its originating `task` by the `replyTo` field. Implementations **MUST** be able to correlate results to tasks. If a task generates multiple partial results, each **MUST** reference the original task `id` in `replyTo` and share the same `correlationId`.

7. Transport Layer

7.1 Transport Requirements

Any LCP transport binding **MUST** satisfy the following requirements:

1. **Encryption**: All data in transit MUST be encrypted with at least 128-bit security.
2. **Authentication**: The transport MUST authenticate both endpoints.
3. **Integrity**: The transport MUST detect message tampering.
4. **Atomicity**: Message delivery MUST be atomic — partial messages MUST NOT be delivered.
5. **Naming**: The transport MUST preserve the original filename.

7.2 Taildrop Binding (Normative)

The normative transport binding for LCP/1.0 uses **Tailscale Taildrop**, which provides peer-to-peer file transfer over WireGuard tunnels.

7.2.1 File Naming Convention

LCP message files **MUST** follow this naming convention:

```
msg_{uuid-prefix-8}.json
```

Where `{uuid-prefix-8}` is the first 8 characters of the message's UUID `id` field.

> **Example:** Message with `id: "550e8400-e29b-41d4-a716-446655440000"` → filename: `msg_550e8400.json`

7.2.2 Send Procedure

1. Construct the LCP envelope as a JSON object.
2. Serialize to UTF-8 JSON with 2-space indentation.
3. Write to a temporary file: `/tmp/msg_{prefix}.json`
4. Archive a copy to `{DATA_DIR}/outbox/msg_{prefix}.json`
5. Execute: `tailscale --socket {SOCKET} file cp {tmpfile} {peer-tailscale-name}:`
6. Remove the temporary file.

7. Verify the exit code. Non-zero indicates delivery failure.

7.2.3 Receive Procedure

1. Execute: `tailscale --socket {SOCKET} file get {INBOX_DIR}/`
2. List all files matching `msg_*.json` in the inbox directory.
3. For each file, in lexicographic order:
    a. Read with `utf-8-sig` encoding (handles Windows BOM).
    b. Parse as JSON.
    c. Validate envelope fields (Section 4).
    d. Verify `from` is in the peer registry at Trust Level ≥ 1.
    e. Check `type` against the sender's trust level (Section 3.3).
    f. Dispatch to the application layer.
    g. Move to `{DATA_DIR}/inbox_archive/`.

> **SECURITY:** The receive procedure MUST use `utf-8-sig` encoding to handle the UTF-8 BOM (byte order mark, `0xEF 0xBB 0xBF`) that Windows implementations may prepend. Failure to handle BOM will cause JSON parse errors.

7.2.4 Taildrop Specifics

Parameter	Value
Socket path (macOS Homebrew)	`/opt/homebrew/var/run/tailscale/tailscaled.sock`
Socket path (Linux)	`/var/run/tailscale/tailscaled.sock`
Socket path (Windows)	Named pipe: `\\.\pipe\ProtectedPrefix\Administrators\Tailscale\tailscaled`
Send timeout	30 seconds
Receive timeout	10 seconds
Encryption	WireGuard (Curve25519, ChaCha20-Poly1305, BLAKE2s)

7.3 HTTP Sync Binding (Informative)

An alternative transport binding using HTTP REST APIs over Tailscale. This is defined informatively for implementations that prefer continuous bidirectional synchronization.

Endpoint	Method	Description
`/api/sync/push`	POST	Push files to peer
`/api/sync/pull`	GET	Pull files from peer
`/api/sync/status`	GET	Check peer online status

HTTP Sync **MUST** use shared-secret authentication (Bearer token) and **MUST** operate over the Tailscale network (ensuring WireGuard encryption).

7.4 Transport Negotiation

LCP/1.0 does not define transport negotiation. If multiple transports are available, implementations **SHOULD** prefer Taildrop for reliability and **MAY** fall back to HTTP Sync if Taildrop is unavailable.

8. Lifecycle & State Machine

8.1 Connection States

```
                    ┌──────────┐
                    │          │
        ┌──────────►│ OFFLINE  │◄──────────┐
        │           │          │           │
        │           └──────────┘           │
        │                │                 │
   peer leaves     peer joins        timeout
   tailnet         tailnet           (72h)
        │                │                 │
        │           ┌────▼─────┐           │
        │           │          │           │
        │           │ ONLINE   │───────────┘
        │           │          │
        │           └──────────┘
        │                │
        │            ping/pong
        │            exchange
        │                │
        │           ┌────▼─────┐
        │           │          │
        └───────────│ TRUSTED  │
                    │          │
                    └──────────┘
```

Figure 4: Peer Connection State Machine

8.2 Message Processing Pipeline

```
┌──────────┐     ┌──────────┐     ┌──────────┐     ┌──────────┐     ┌──────────┐
│ Transport│     │  Parse   │     │ Validate │     │  Trust   │     │ Dispatch │
│  Receive │──►  │  JSON    │──►  │ Envelope │──►  │  Check   │──►  │ to App   │
│          │     │          │     │          │     │          │     │          │
└──────────┘     └──────────┘     └──────────┘     └──────────┘     └──────────┘
                     │                 │                 │                │
                parse error     invalid fields     untrusted        success
                     │                 │                 │                │
                     ▼                 ▼                 ▼                ▼
                  ┌──────┐          ┌──────┐          ┌──────┐         ┌──────┐
                  │ Log  │          │ Log  │          │ Drop │         │Archive│
                  │ Warn │          │Error │          │ Warn │         │ Move  │
                  └──────┘          └──────┘          └──────┘         └──────┘
```

Figure 5: Message Processing Pipeline

8.3 Inbox / Outbox Model

Each Lobster maintains a local data directory with the following structure:

```
{SKILL_DIR}/
├── data/
│   ├── inbox/            # Incoming messages (pending processing)
│   ├── inbox_archive/    # Processed incoming messages
│   └── outbox/           # Copies of sent messages
├── config/
│   └── peers.json        # Peer registry
└── scripts/
    ├── send.py           # Send implementation
    ├── recv.py           # Receive implementation
    ├── check_and_notify.py  # Non-archiving inbox check
    └── ack.py            # Manual archive trigger
```

8.4 Archival

LCP mandates a **two-phase receive** model:

1. **Check Phase**: Fetch and read messages without archiving. This allows the application layer to read and process messages without risk of data loss.
2. **Acknowledge Phase**: After the application has fully processed and responded to a message, it is moved to the archive directory.

This two-phase model prevents the "swallowed message" anti-pattern where messages are archived before being processed, leading to lost messages if the processing step fails.

> **NOTE:** Archival is a local operation. It does not send any network message. The `ack` message type (Section 5.5) is a separate, optional protocol-level acknowledgment.

9. Reliability & Delivery Guarantees

9.1 At-Least-Once Delivery

LCP provides **at-least-once** delivery semantics. The transport layer (Taildrop) guarantees that a successfully sent file will eventually arrive at the recipient, but the same message **MAY** be delivered more than once in certain failure scenarios (e.g., sender crashes after Taildrop succeeds but before recording success locally).

9.2 Idempotency

Receivers **MUST** implement idempotency based on the message `id` field:

- Maintain a set of recently processed message IDs (minimum: last 1000 IDs or last 24 hours).
- If a received message's `id` matches a previously processed ID, discard it silently.
- The deduplication window **SHOULD** be at least as long as the default TTL (24 hours).

9.3 Ordering

LCP does **not** guarantee message ordering. Messages **MAY** arrive out of order due to transport scheduling, retries, or network conditions. Applications that require ordering **MUST** use the `timestamp` field to reconstruct chronological order and **SHOULD** use `correlationId` to group related messages.

9.4 Retry Semantics

If the transport layer reports a delivery failure:

1. The sender **SHOULD** retry with exponential backoff.
2. Initial retry delay: 5 seconds. Maximum delay: 5 minutes. Maximum retries: 10.
3. After exhausting retries, the sender **MUST** log the failure and **MAY** notify the Principal.
4. The sender **MUST NOT** generate a new `id` for retried messages.

10. Error Handling

10.1 Error Codes

Code	Name	Description
E001	PARSE_ERROR	JSON parsing failed
E002	INVALID_ENVELOPE	Required field missing or malformed
E003	UNKNOWN_SENDER	Sender not in peer registry
E004	TRUST_VIOLATION	Message type not allowed at sender's trust level
E005	TTL_EXPIRED	Message exceeded its time-to-live
E006	DUPLICATE_ID	Message ID already processed (deduplication)
E007	SIZE_EXCEEDED	Envelope exceeds maximum size
E008	TASK_FAILED	Task could not be completed
E009	TASK_TIMEOUT	Task exceeded its deadline
E010	VERSION_MISMATCH	Unsupported protocol version
E011	TRANSPORT_ERROR	Transport-level failure
E099	INTERNAL_ERROR	Unspecified internal error

10.2 Error Message Format

```
{
  "lcp": "1.0",
  "id": "...",
  "from": "CloudJeffreyBOT",
  "to": "JeffreyBOT",
  "timestamp": "...",
  "type": "error",
  "message": "Task failed: unable to access the requested resource.",
  "replyTo": "7344f936-...",
  "extensions": {
    "lcp-error": {
      "code": "E008",
      "name": "TASK_FAILED",
```

```
      "detail": "HTTP 403 received when accessing stock price API.",
      "retryable": false
    }
  }
}
```

10.3 Recovery Procedures

Error	Sender Action	Receiver Action
E001, E002	Fix message format, resend	Log and discard
E003, E004	N/A (configuration issue)	Log and discard; notify principal
E005	Resend if still relevant	Discard expired message
E006	No action needed	Discard duplicate silently
E008, E009	Retry or reassign task	Send error report
E010	Negotiate version or upgrade	Send error report
E011	Retry with backoff	N/A

11. Security Considerations

11.1 Transport Security

LCP **REQUIRES** encrypted transport. The normative Taildrop binding inherits WireGuard's security properties:

- **Key Exchange**: Curve25519 Diffie-Hellman
- **Encryption**: ChaCha20-Poly1305 (AEAD)
- **Hashing**: BLAKE2s
- **Perfect Forward Secrecy**: Yes (keys rotate every 2 minutes)

LCP messages **MUST NOT** be transmitted over unencrypted channels. Implementations **MUST** verify that the transport provides at least 128-bit security before sending messages.

11.2 Identity Verification

Identity verification relies on the Tailscale control plane:

- Each device on the Tailnet has a unique WireGuard public key bound to an authenticated identity.
- Taildrop verifies the sender's identity at the Tailscale level before accepting file transfers.
- LCP adds an application-level check: the `from` field **MUST** match the expected LName for the Tailscale peer that delivered the file.

> **SECURITY:** Implementations MUST validate that the `from` field matches the peer registry entry for the delivering Tailscale node. Accepting a `from` field without cross-referencing the transport-level identity enables identity spoofing attacks.

11.3 Message Integrity

Message integrity is provided at two levels:

1. **Transport Level**: WireGuard's Poly1305 MAC ensures that messages are not tampered with in transit.
2. **Application Level (OPTIONAL)**: Implementations **MAY** include a HMAC-SHA256 signature in `extensions["lcp-integrity"]` for defense-in-depth.

```
"extensions": {
  "lcp-integrity": {
    "algorithm": "HMAC-SHA256",
    "signature": "base64-encoded-hmac",
    "keyId": "shared-key-identifier"
  }
}
```

11.4 Confidentiality

LCP messages are stored as plaintext JSON on the local filesystem. This is by design (auditability goal). Implementations **SHOULD**:

- Set file permissions to `0600` (owner read/write only) for inbox and outbox directories.
- Use encrypted filesystems (FileVault, BitLocker, LUKS) for the data directory.
- Consider application-level encryption for messages containing sensitive content, using the `lcp-encrypted` extension.

11.5 Replay Protection

Replay attacks are mitigated by:

- **UUID uniqueness**: Each message has a unique ID; deduplication (Section 9.2) prevents replayed messages from being processed twice.
- **TTL enforcement**: Expired messages are discarded.
- **Timestamp validation**: Implementations **SHOULD** reject messages with timestamps more than `ttl` seconds in the past.

11.6 Denial of Service

Potential DoS vectors and mitigations:

Vector	Mitigation
Message flood	Rate limiting: max 100 messages per peer per hour
Large message	Size limit enforcement (1 MiB max)
Malformed JSON	Parse with size-limited reader; reject on first error
Disk exhaustion	Maximum inbox size: 100 MiB; oldest messages purged

11.7 Privacy

LCP messages may contain private information from the Principal's environment. Implementations:

- **MUST NOT** forward message contents to third parties without Principal consent.
- **MUST NOT** log message contents to external services.
- **SHOULD** implement configurable retention policies for archived messages.
- **MUST** delete archived messages after a configurable retention period (default: 30 days).

12. Encoding & Interoperability

12.1 Character Encoding

- All LCP messages **MUST** be encoded in UTF-8 (RFC 3629).
- Receivers **MUST** accept UTF-8 with or without BOM.
- Senders **SHOULD NOT** include BOM.
- JSON serialization **MUST** use `ensure_ascii=False` (or equivalent) to preserve Unicode characters.

12.2 Timestamp Format

- Timestamps **MUST** conform to RFC 3339 / ISO 8601.
- Timestamps **MUST** include timezone offset (e.g., `+08:00`, `Z`).
- Sub-second precision **SHOULD** be microsecond (6 digits).
- Example: `2026-03-26T13:56:00.518245+08:00`

12.3 UUID Format

- Message IDs **MUST** be UUID version 4, lowercase, with hyphens (RFC 9562).
- Example: `550e8400-e29b-41d4-a716-446655440000`

12.4 Cross-Platform Considerations

Issue	Recommendation
Line endings	Use `\n` (LF). Accept `\r\n` (CRLF).
File paths	Use forward slash `/` in all path references.
BOM	Read with `utf-8-sig`; write without BOM.
JSON indentation	2 spaces (for human readability). Receivers MUST accept any valid JSON.
Filename case	Treat as case-sensitive. Use lowercase `msg_` prefix.

13. Extensibility

13.1 Extension Fields

The `extensions` object provides a namespace-keyed mechanism for extending the envelope without modifying the core schema:

```
"extensions": {
  "lcp-task": { "deadline": "...", "category": "..." },
  "lcp-integrity": { "algorithm": "...", "signature": "..." },
  "x-myapp-metadata": { "version": "2.0", "source": "cron" }
}
```

Rules:

- Standard extensions use the `lcp-` prefix.
- Custom extensions **MUST** use the `x-` prefix.
- Receivers **MUST** ignore unknown extensions without error.
- Extensions **MUST NOT** duplicate or override core envelope fields.

Registered Standard Extensions

Namespace	Purpose	Status
`lcp-task`	Task metadata (deadline, category, priority details)	Stable
`lcp-result`	Structured result data	Stable
`lcp-error`	Structured error details (code, name, retryable)	Stable
`lcp-integrity`	HMAC signature for defense-in-depth	Experimental
`lcp-encrypted`	Application-level payload encryption	Experimental
`lcp-routing`	Multi-hop routing metadata (LCP/2.0)	Draft

13.2 Custom Message Types

Custom message types **MUST** use the `x-` prefix:

- `x-heartbeat` — Periodic liveness check

- `x-file-transfer` — Large file transfer coordination
- `x-broadcast` — Multi-peer announcement (LCP/2.0 preview)

Receivers that do not understand a custom type **MUST** log a warning and **MAY** respond with an `error` (code E002) or silently discard the message.

13.3 Version Negotiation

If a receiver receives a message with an unsupported `lcp` version:

1. It **MUST** respond with an `error` message (code E010).
2. The error's `extensions["lcp-error"]` **SHOULD** include a `supportedVersions` array.
3. The sender **MAY** retry with a supported version.

14. IANA-Like Considerations

This specification defines the following registries, maintained by the Lobster Communication Working Group:

Registry	Contents	Registration Policy
LCP Message Types	Core and extension message types	Standards Action (core), First Come First Served (x-)
LCP Error Codes	Error code numbers and names	Standards Action
LCP Extension Namespaces	Standard extension prefixes	Expert Review

15. References

15.1 Normative References

[RFC2119]	Bradner, S., "Key words for use in RFCs to Indicate Requirement Levels", BCP 14, RFC 2119, March 1997.
[RFC3339]	Klyne, G. and C. Newman, "Date and Time on the Internet: Timestamps", RFC 3339, July 2002.
[RFC3629]	Yergeau, F., "UTF-8, a transformation format of ISO 10646", STD 63, RFC 3629, November 2003.
[RFC8174]	Leiba, B., "Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words", BCP 14, RFC 8174, May 2017.
[RFC8259]	Bray, T., Ed., "The JavaScript Object Notation (JSON) Data Interchange Format", STD 90, RFC 8259, December 2017.
[RFC9562]	Davis, K., Peabody, B., and P. Leach, "Universally Unique IDentifiers (UUIDs)", RFC 9562, May 2024.

15.2 Informative References

[WIREGUARD]	Donenfeld, J.A., "WireGuard: Next Generation Kernel Network Tunnel", 2017. https://www.wireguard.com/papers/wireguard.pdf
[TAILSCALE]	Tailscale Inc., "How Tailscale Works", 2024. https://tailscale.com/blog/how-tailscale-works
[TAILDROP]	Tailscale Inc., "Taildrop", 2024. https://tailscale.com/kb/1106/taildrop
[NOISE]	Perrin, T., "The Noise Protocol Framework", Revision 34, 2018. https://noiseprotocol.org/noise.html
[JSONSCHEMA]	Wright, A. et al., "JSON Schema: A Media Type for Describing JSON Documents", Internet-Draft, 2022.

Appendix A — Complete JSON Schema

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://lobster-comm.protocol/schemas/lcp-envelope-1.0.json",
  "title": "LCP/1.0 Message Envelope",
  "description": "Lobster Communication Protocol message envelope schema
  "type": "object",
  "required": ["lcp", "id", "from", "to", "timestamp", "type", "message"
  "properties": {
    "lcp": {
      "type": "string",
      "const": "1.0",
      "description": "Protocol version"
    },
    "id": {
      "type": "string",
      "format": "uuid",
      "description": "Unique message identifier (UUID v4)"
    },
    "from": {
      "type": "string",
      "pattern": "^[A-Za-z][A-Za-z0-9_-]{0,63}$",
      "description": "Sender Lobster Name"
    },
    "to": {
      "type": "string",
      "pattern": "^[A-Za-z][A-Za-z0-9_-]{0,63}$",
      "description": "Recipient Lobster Name"
    },
    "timestamp": {
      "type": "string",
      "format": "date-time",
      "description": "ISO 8601 timestamp with timezone"
    },
    "type": {
      "type": "string",
      "enum": ["ping", "pong", "chat", "task", "result", "ack", "error"]
      "description": "Message type (core types; x- prefixed types also v
    },
    "message": {
      "type": "string",
      "maxLength": 65536,
      "description": "Human-readable message body"
    },
```

```
    "replyTo": {
      "type": ["string", "null"],
      "format": "uuid",
      "description": "UUID of the message being replied to"
    },
    "correlationId": {
      "type": ["string", "null"],
      "format": "uuid",
      "description": "Conversation thread identifier"
    },
    "ttl": {
      "type": ["integer", "null"],
      "minimum": 0,
      "maximum": 604800,
      "default": 86400,
      "description": "Time-to-live in seconds"
    },
    "priority": {
      "type": "string",
      "enum": ["low", "normal", "high", "urgent"],
      "default": "normal"
    },
    "extensions": {
      "type": "object",
      "additionalProperties": { "type": "object" },
      "description": "Namespace-keyed extension data"
    }
  },
  "additionalProperties": false
}
```

Appendix B — Reference Implementation

The following Python 3 code provides a minimal, spec-compliant LCP/1.0 implementation. It requires only Python standard library modules.

B.1 Envelope Construction

```
import json, uuid
from datetime import datetime, timezone, timedelta

def create_envelope(from_name, to_name, msg_type, message,
                    reply_to=None, correlation_id=None,
                    ttl=86400, priority="normal", extensions=None):
    """Create a spec-compliant LCP/1.0 envelope."""
    tz = timezone(timedelta(hours=8))  # Configurable
    return {
        "lcp": "1.0",
        "id": str(uuid.uuid4()),
        "from": from_name,
        "to": to_name,
        "timestamp": datetime.now(tz).isoformat(),
        "type": msg_type,
        "message": message,
        "replyTo": reply_to,
        "correlationId": correlation_id,
        "ttl": ttl,
        "priority": priority,
        "extensions": extensions or {}
    }
```

B.2 Envelope Validation

```
import re

LNAME_RE = re.compile(r'^[A-Za-z][A-Za-z0-9_-]{0,63}$')
UUID_RE = re.compile(
    r'^[0-9a-f]{8}-[0-9a-f]{4}-4[0-9a-f]{3}-[89ab][0-9a-f]{3}-[0-9a-f]{1
)
VALID_TYPES = {"ping","pong","chat","task","result","ack","error"}
MAX_SIZE = 1_048_576  # 1 MiB

def validate_envelope(raw_bytes):
    """Validate an LCP envelope. Returns (envelope, errors)."""
```

```
    errors = []
    if len(raw_bytes) > MAX_SIZE:
        return None, ["E007: Envelope exceeds 1 MiB"]
    try:
        env = json.loads(raw_bytes.decode('utf-8-sig'))
    except (json.JSONDecodeError, UnicodeDecodeError) as e:
        return None, [f"E001: Parse error: {e}"]

    for field in ("lcp", "id", "from", "to", "timestamp", "type", "messa
        if field not in env:
            errors.append(f"E002: Missing required field: {field}")
    if errors:
        return None, errors

    if env["lcp"] != "1.0":
        errors.append(f"E010: Unsupported version: {env['lcp']}")
    if not UUID_RE.match(env.get("id", "")):
        errors.append("E002: Invalid UUID format for 'id'")
    if not LNAME_RE.match(env.get("from", "")):
        errors.append("E002: Invalid LName for 'from'")
    if not LNAME_RE.match(env.get("to", "")):
        errors.append("E002: Invalid LName for 'to'")

    msg_type = env.get("type", "")
    if msg_type not in VALID_TYPES and not msg_type.startswith("x-"):
        errors.append(f"E002: Unknown message type: {msg_type}")

    return env, errors
```

B.3 Transport: Taildrop Send

```
import subprocess, tempfile, os

def taildrop_send(envelope, peer_tailscale_name, socket_path, outbox_dir
    """Send an LCP envelope via Taildrop. Returns success boolean."""
    fname = f"msg_{envelope['id'][:8]}.json"
    payload = json.dumps(envelope, ensure_ascii=False, indent=2)

    # Archive locally
    os.makedirs(outbox_dir, exist_ok=True)
    with open(os.path.join(outbox_dir, fname), 'w', encoding='utf-8') as
        f.write(payload)

    # Send via Taildrop
    tmp = os.path.join(tempfile.gettempdir(), fname)
    with open(tmp, 'w', encoding='utf-8') as f:
        f.write(payload)
```

```
    try:
        result = subprocess.run(
            ["tailscale", "--socket", socket_path,
             "file", "cp", tmp, f"{peer_tailscale_name}:"],
            capture_output=True, text=True, timeout=30
        )
        return result.returncode == 0
    finally:
        if os.path.exists(tmp):
            os.remove(tmp)
```

Appendix C — Example Message Flows

C.1 Handshake Flow

```
JeffreyBOT                                    CloudJeffreyBOT
   |                                                  |
   |   ping: "Hello from the Mac side! 🦞"            |
   |──────────────────────────────────────────────────►|
   |                                                  |
   |   pong: "Pong! Windows side ready. 🦞"           |
   |◄──────────────────────────────────────────────────|
   |                                                  |
   |   [Trust established: bidirectional]              |
```

C.2 Task Delegation Flow

```
JeffreyBOT                                    CloudJeffreyBOT
   |                                                  |
   |   task: "Check AAPL stock price"                 |
   |   correlationId: "τ"                             |
   |──────────────────────────────────────────────────►|
   |                                                  |
   |   ack: "Task received, processing."              |
   |   replyTo: task.id, correlationId: "τ"           |
   |◄──────────────────────────────────────────────────|
   |                                                  |
   |            ... (processing time) ...             |
   |                                                  |
   |   result: "AAPL: $247.50 (+1.2%)"               |
   |   replyTo: task.id, correlationId: "τ"           |
   |◄──────────────────────────────────────────────────|
```

C.3 Error Recovery Flow

```
JeffreyBOT                                    CloudJeffreyBOT
   |                                                  |
   |   task: "Access restricted database"             |
   |──────────────────────────────────────────────────►|
   |                                                  |
   |   error: E008 TASK_FAILED                        |
   |   "HTTP 403 - access denied"                     |
   |   retryable: false                               |
   |◄──────────────────────────────────────────────────|
   |                                                  |
   |   [JeffreyBOT logs error, notifies Principal]    |
```

C.4 Conversation Thread Flow

```
JeffreyBOT                                    CloudJeffreyBOT
   |                                                  |
   |   chat(A): "1+3=?" correlationId=A               |
   |──────────────────────────────────────────────────►|
   |                                                  |
   |   chat(B): "4!" replyTo=A, correlationId=A       |
   |◄──────────────────────────────────────────────────|
   |                                                  |
   |   chat(C): "Correct ✅" replyTo=B, corr=A        |
   |──────────────────────────────────────────────────►|
```

Appendix D — Change Log

Version	Date	Changes
1.0.0	2026-03-26	Initial release. Core protocol specification based on the operational Lobster-Comm system deployed between JeffreyBOT (macOS) and CloudJeffreyBOT (Windows).

LCP-SPEC-2026-0001 | Lobster Communication Protocol 1.0 | March 2026

🦞 End of Document 🦞